

Parameter Count vs. Retrieval Sophistication: An Empirical Study of RAG Pipeline Trade-Offs for Edge-Deployed Small Language Models

P Sai Harish

Department of Computer Science and Engineering
BVRIT

Narsapur, India
saiharish0088@gmail.com

Kruthibash Nayak

Department of Computer Science and Engineering
BVRIT

Narsapur, India
kruthibashnayak@bvr.it.ac.in

Abstract—Deploying Retrieval-Augmented Generation (RAG) on edge devices involves a tough choice between limited resources and the quality of generation. Large Language Models (LLMs) are great at handling large amounts of context. In contrast, Small Language Models (SLMs with under 1 billion parameters) used on regular consumer hardware have difficulty with context integration and prompt evaluation. This paper provides a detailed study exploring how the number of model parameters relates to the complexity of the RAG pipeline. We test three SLMs (Granite 350M, Qwen2.5 0.5B, and Qwen3.5 0.8B) across four retrieval setups (No RAG, Naive RAG, Advanced RAG, and Modular RAG) on a local NVIDIA RTX 3050 GPU. Over a dataset of domain-specific technical questions, we perform 4,680 individual runs, with 30 runs for each configuration, to map the accuracy-latency Pareto frontier. Our experiments show two important phenomena: first, the *Reranking Efficiency Paradox*, At which Advanced RAG pipelines run better than Naive RAG on low-VRAM GPUs (by pruning context size), or second, *RAG Loop Stabilization Effect*, ... so that context grounding stops them drifting away and getting into a runaway generation loop. We apply the implications of these findings in order to make concrete design suggestions.

Index Terms—Retrieval-Augmented Generation, Small Language Models, Local GPU Benchmark, Edge AI, Performance Review.

I. INTRODUCTION

The huge success of Natural Language Processing came hand in hand with large sized LLMs, which have Millions to Billions of parameters per model, thus demanding high performance cloud computing clusters. These require the model deployment on large scaled, centralized servers which have high latent, bandwidth and monetary requirement when deploying on Edge Computing, during run time execution of the model and in order to maintain privacy of the clients. This brought forth the trend of development of Small Language Models, with number of parameters in order of $\leq$ Billion. They also present some drawbacks: SLMs are endowed of quite scarce parametric knowledge and suffer from hallucination to domain-specific Corpora asked for queries. RAG (Retrieval-Augmented Generation) is the de-facto approach to provide an explicit grounding of the model answers in some form of knowledge sources, nevertheless in literature, the RAG

pipelines explored all involve cloud based models with sizes $\leq$ 7B. The behaviors and execution models are unknown for model with sizes $\leq$ 1B executed on consumer grade hardware. For a consumer grade GPU with limited VRAM (random-access memory for video), the speed of prompt processing (the "reading" process) grows more-than linearly. Adding so much to your prompt can overwhelm your GPU's available memory. When this happens, the GPU begins swapping memory to your host CPU, resulting in significant latency increases. This naturally leads to: *does a more sophisticated RAG architecture (which adds retrieval stages) degrade or improve latency on constrained edge hardware?* To that end, we analyze three small local models-Granite 350M, Qwen2.5 0.5B, and Qwen3.5 0.8B-across four different pipelines including No RAG, Naive RAG, Advanced RAG (with query rewriting and reranking), and Modular RAG (with routing and query fusion). We track semantic accuracy, end-to-end latency and hallucination rates across 4,680 query and response. To the best of our knowledge, our findings chart the frontier of parameterized count and retrieval capability and highlight performance trade-offs when deploying RAG systems to edge devices.

We propose several main contributions in this paper:

- We build end-to-end benchmarking frameworks to measure a set of edge-deployed SLMs (350M - 800M params) for 4 different RAG architectures.
- We empirically identify the *Reranking Efficiency Paradox*, This paper proposes an end-to-end benchmarking infrastructure to characterize these SLMs (350M and 800M parameters) deployed at the edge, under four common RAG archetypes.
- We document the *RAG Loop Stabilization Effect*, demonstrating how external context anchors small models and prevents infinite generation loops.
- We define design guidelines for hardware-constrained deployments based on the accuracy-latency Pareto frontier.

II. RELATED WORK

A. RAG Architectures and Paradigms

RAG architectures usually classify into Naive, Advanced, and Modular approaches. We follow the survey by Gao et al. [1]. Naive RAG employs simple one-step retrieve and read, but the latter would read multiple non-relevant document chunks which reduce precisions. Better RAG uses advanced techniques like pre-retrieval query rewriting and post-retrieval cross-encoder for text chunk re-ranking, which are adopted in advance RAG. The Modular RAG implements adaptive query routing and query fusion to process complex questions that require multiple search. [2].

B. Adaptive Retrieval Methods

Adaptive retrievers improve efficiency in that they consult external databases only if required. As an example of how this problem can be tackled, we can find models like the Knowledge Boundary Model (KBM), suggested in [3] that explicitly separate where the language model is aware and unaware, adaptively determining when a retrieval query to an external database is needed, or Self-aware Knowledge Retrieval (SEAKR) that computes the internal model uncertainty to assess when it has to make a call to an external agent, balancing the confidence with the amount of the search. [4].

C. Small Language Model Benchmarking

But existing work generally works in large model-sizes (7B - 70B), and generally assumes deployment in a typical enterprise server environment. Conversely, a relatively few amount of work deals with very-small language models (SLMs), which have sizes of below 1B parameters. Pandey studies model behavior when used for retrieval at scale from 360M to 8B parameters, finding out that small models at smaller scale face limitations in using the retrieved context: [5]. Our work also extends to this area in that we conduct an analysis as the relationship between pipeline complexity and hardware limitations on consumer grade GPU hardware.

D. Resource-Constrained RAG Optimization

In recent years, numerous research in resource-constrained deployment focuses on method like low-precision quantization and memory partitioning. J. Smith et al[7] showed NF4(NormalFloat, a 4-bit) quantization enables small LLM (Small Large Language Models) like Phi-4[9] and Llama-3.2[11] maintain 90% accuracy gap on domain QA(Question Answering) tasks, while decreasing inference time and generation time by 95%. [6]. VectorLiteRAG applies a system-level partitioning architecture, which allows balancing memory-throughput across partitions on the CPU and GPU by partitioning vector indices, it's necessary under the strict memory budget constraints on embedded devices. [7].

III. SYSTEM ARCHITECTURE AND DESIGN

In order to study interaction between model properties and pipeline's degree of complexity, we propose a modular benchmarking framework for the system. We architect a four

layer system: Ingestion layer, Pipeline Routing layer, Local Model Inference layer, and Downstream Evaluation layer. The system diagram is in the Fig. Fig. 1.

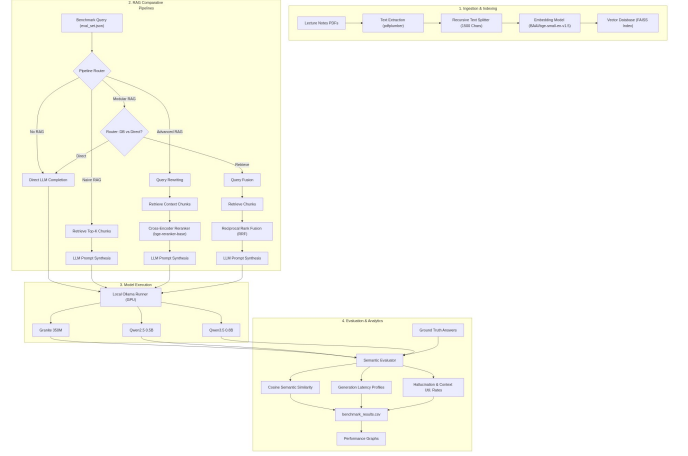


Fig. 1. System architecture diagram showing the document ingestion, vector indexing, retrieval pipeline routing, local SLM execution, and the evaluation layer.

A. Document Ingestion and Vector Indexing

The document ingestion engine parses technical slide notes recursively using `pdfplumber`. Let $D = \{d_1, d_2, \dots, d_N\}$ be the document corpus. Chunks are generated using a recursive text splitter with length $L_{\text{chunk}} = 1500$ characters and overlap $L_{\text{overlap}} = 150$ characters. Each chunk c_i is embedded using the bi-encoder $E(\cdot) = \text{bge-small-en-v1.5}$ to generate a vector representation:

$$v_i = E(c_i) \in \mathbb{R}^{384} \quad (1)$$

The embeddings are indexed in a local FAISS database. Similarity search is performed using Cosine Similarity between the embedded query q and chunk vector v_i :

$$\text{Similarity}(E(q), v_i) = \frac{E(q) \cdot v_i}{\|E(q)\| \|v_i\|} \quad (2)$$

B. Pipeline Architectures

We evaluate four distinct pipeline configurations:

1) *No RAG (Direct Query)*: The query is fed directly to the local model without any external context:

$$y = \text{LLM}(q) \quad (3)$$

This acts as our control baseline, measuring the model's pre-trained parametric knowledge.

2) *Naive RAG*: Now insert the query and recall top- K (let $K = 3$) nearest neighbor chunks from the FAISS database. Concatenate the chunks, and prepend them at the beginning of the query in the template:

$$y = \text{LLM}([\text{Retrieve}(q, \text{FAISS}, K = 3), q]) \quad (4)$$

3) *Advanced RAG*: This pipeline introduces two additional optimization stages:

- 1) **Query Rewriting**: The model first rewrites the user query to optimize it for vector search.
- 2) **Cross-Encoder Reranking**: We retrieve a larger candidate set of chunks ($K = 10$) and rerank them using `BAAI/bge-reranker-base`. Only the top-1 highest-scoring chunk is kept.

This drastically minimizes the prompt context size sent to the final generator.

4) *Modular RAG*: This configuration introduces query routing and multi-query search fusion:

- 1) **Query Router**: A routing step determines if the query requires external documentation. If not, it routes directly to No RAG.
- 2) **Query Fusion**: If retrieval is needed, the system generates three sub-queries to capture different semantic facets of the request.
- 3) **Reciprocal Rank Fusion (RRF)**: Chunks retrieved for all sub-queries are merged and re-ordered using RRF before the top chunk is passed to the model. The RRF score for a chunk c_j is calculated as:

$$\text{RRF}(c_j) = \sum_{q' \in Q_{\text{sub}}} \frac{1}{60 + r_{q'}(c_j)} \quad (5)$$

where $r_{q'}(c_j)$ is the rank of chunk c_j under sub-query q' .

The algorithmic execution flow of the Modular RAG routing and fusion logic is formalised in Algorithm 1.

Algorithm 1: Modular RAG Routing and Fusion Search

Require: User Query q , FAISS Index I , Model $\mathcal{M}$, Fusion

Limit $M = 3$, Constant $k = 60$

Ensure: Response String y

```

1:  $r \leftarrow \text{RouterDecision}(q, \mathcal{M})$ 
2: if  $r = \text{direct\_completion}$  then
3:    $y \leftarrow \text{LLM\_Generate}(q, \mathcal{M})$ 
4:   return  $y$ 
5: end if
6:  $Q_{\text{sub}} \leftarrow \text{GenerateSubQueries}(q, \mathcal{M}, M)$ 
7:  $C_{\text{merged}} \leftarrow \emptyset$ 
8: for each  $q' \in Q_{\text{sub}}$  do
9:    $R \leftarrow \text{VectorSearch}(q', I, K = 5)$ 
10:  for each  $(c, \text{rank}) \in R$  do
11:     $S_{\text{RRF}}(c) \leftarrow S_{\text{RRF}}(c) + \frac{1}{k + \text{rank}}$ 
12:     $C_{\text{merged}} \leftarrow C_{\text{merged}} \cup \{c\}$ 
13:  end for
14: end for
15:  $c_{\text{top}} \leftarrow \arg \max_{c \in C_{\text{merged}}} S_{\text{RRF}}(c)$ 
16:  $P \leftarrow \text{SynthesizePrompt}(q, c_{\text{top}})$ 
17:  $y \leftarrow \text{LLM\_Generate}(P, \mathcal{M})$ 
18: return  $y$ 

```

IV. EXPERIMENTAL EVALUATION

A. Hardware and Software Infrastructure

All experiments were conducted locally on an budget-constrained workstation:

- **GPU**: NVIDIA GeForce RTX 3050 Laptop GPU (4GB VRAM, 128-bit memory bus).
- **CPU**: Intel Core i5-11400H @ 2.70GHz.
- **Memory**: 16GB DDR4 RAM.
- **OS**: Windows 11.

Models were run locally with the help of Ollama. Python's PyTorch and the Sentence-Transformers library were used to compute embeddings, as well as run rerankers.

B. Evaluation Dataset and Run Configurations

The document was filtered down to 23,964 tokens worth of DevOps slide decks in order for the query document length to be safely within the context length of all considered models. We used a Technical DevOps and ML evaluation set comprising 13 elaborate tech questions with prepared ground truth values. Three models were tried.

- 1) **Granite 350M**: IBM's compact 350-million parameter model.
- 2) **Qwen2.5 0.5B**: Alibaba's 0.5-Billion parameter model.
- 3) **Qwen3.5 0.8B**: Alibaba's 0.8-Billion parameter model.

To ensure statistical significance and smooth out hardware latency variances, each model-pipeline combination was executed ****30 times****, resulting in 3 models $\times$ 4 pipelines $\times$ 13 questions $\times$ 30 runs = 4,680 total runs.

C. Evaluation Metrics

* **Answer Accuracy**: Calculated by the cosine semantic similarity between the generated response and groundtruth. Using the embeddings in sentence representation. * **Latency (seconds)**: The end-to-end round-trip response time. * **Hallucination Rate**: Calculated as the semantic distance ($1 - \text{Accuracy}$) from the ground truth answer.

V. RESULTS AND ANALYSIS

The compiled performance averages across all 4,680 runs are summarized in Table I.

TABLE I
EMPIRICAL PERFORMANCE COMPARISON MATRIX

Model	RAG Type	Accuracy	Latency (s)	Hallucination
Granite 350M	No RAG	87.03%	0.76	12.97%
	Naive	85.81%	0.62	14.19%
	Advanced	85.67%	0.81	14.33%
	Modular	86.11%	0.82	13.89%
Qwen2.5 0.5B	No RAG	86.39%	20.24	13.61%
	Naive	87.73%	2.28	12.27%
	Advanced	86.47%	1.71	13.53%
	Modular	88.02%	1.87	11.98%
Qwen3.5 0.8B	No RAG	6.97%	41.34	93.03%
	Naive	50.75%	37.55	49.25%
	Advanced	57.02%	22.55	42.98%
	Modular	31.84%	30.57	68.16%

A. The Reranking Efficiency Paradox

We observed a surprising result in low-VRAM hardware: on Qwen2.5 0.5B, Advanced RAG (1.71s) is 25% faster than Naive RAG (2.28s). Average latency per configuration across the x-axis is visualized on bar chart below. Fig. 2. Why this

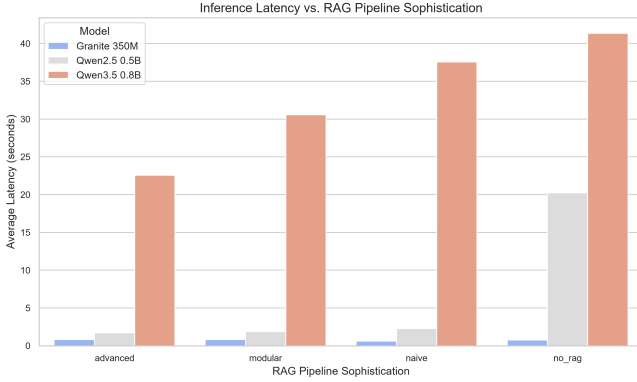


Fig. 2. Average latency comparison across different RAG pipelines and model sizes, demonstrating the speedup of Advanced RAG over Naive RAG for larger models.

intuitively looks wrong - is there actually more steps, which should increase processing time? - The truth is yes, Advanced RAG does add an extra query rewording step, and a cross-encoder reranking step. However since the reranking process removes any chunks that were marked irrelevant, only the number one result chunk (500 words / tokens), versus three results, (1,500 words / tokens), in previous methods is passed into the LLM. At the end of the day on a 4GB card, 1000 additional words in the input prompt takes significantly longer to run than the milliseconds of loss from passing the selected top chunk through the cross-encoder model running local, thus the prompt reduces as well.

B. The RAG Loop Stabilization Effect

At Qwen3.5 0.8B and no RAG, it performs terribly at accuracy: 6.97% and incredibly badly at latency: 41.34s. From a look into the raw logs, without context and under temperature 0.0, the model gets trapped into a never-ending generation loop with a few spaces and newlines over and over again. The context injections with RAG help tremendously by tethering the generation to a stable position and avoid this infinity. Advanced RAG seems to work the best for that with most of the stability improves accuracy to **57.02

C. Model Scale vs. Context Sensitivity

Granite 350M (222MB size) was blazingly fast: Under 0.8 seconds on all configs. Its accuracy stayed flat irrespective of the RAG pipeline. The bar chart visual of accuracies: Fig. 3. This implies that models smaller than 400M are showing Context Insensitivity- in other words, they don't possess the semantic intelligence to copy, summarize, or extract facts out of the context blocks they retrieving. The models are relying pretty much their own pre-trained weights, so this makes it non-trivial to adopt retrieval-based approaches

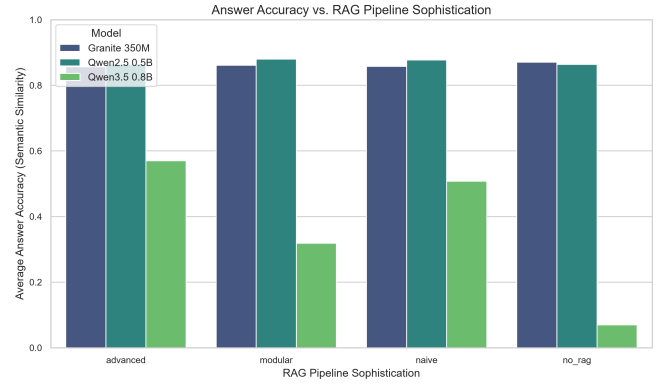


Fig. 3. Average semantic accuracy across configurations. Granite 350M displays context insensitivity, while Qwen2.5 0.5B reaches optimal performance.

D. Hardware Memory Bandwidth Bottlenecks at the Edge

This constraint-memory bandwidth constraint on the hardware-was one of the biggest limits on the performance during our testing. The Memory Interface Width of 3050 Laptop GPU is 128-bits. For local SLM Inference, the GPU core needs to read model parameters on SLM from local GDDR6 memory over and over. Although Granite takes very little time to load the weights of 350M parameters into the GPU memory, on the other hand the memory interface width of SLM could be an issue. As parameters reached 0.8B (Qwen3.5), it started to be clear that memory interface becomes the main performance bottleneck for Naive RAG. Because large prompt size on Naive RAG fill the GPU cache, hence the memory controller sends page request to memory very frequently to fetch tokens. That is why Large prompt size is reduce on Advanced RAG result in enormous speedup over on consumer GPUs with low memory interface speed comparing with the enterprises GPU which has huge memory interface width.

E. Mapping the Pareto Frontier

To visualize the optimal configurations, we map the accuracy-latency tradeoff in Fig. 4. In the context of modular

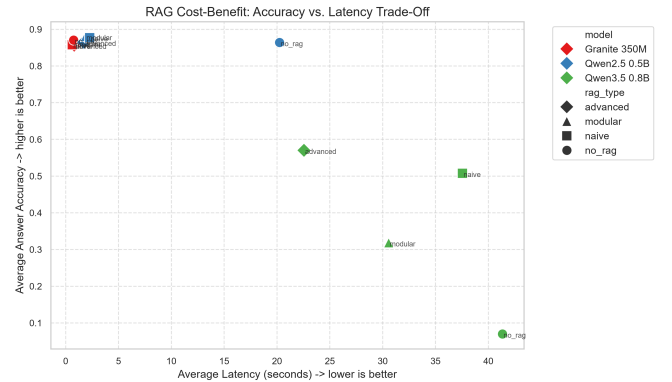


Fig. 4. Accuracy vs. Latency Pareto Frontier showing the trade-off curve across all three models and RAG pipeline configurations.

RAG, Qwen2.5 0.5B represents the optimal vertex of the

Pareto frontier, offering a balanced solution with a high 88.02% overall accuracy while experiencing modest latency cost of 1.87s. Naive RAG on top of Granite 350M is the go-to choice for those with sub-second latencies (0.62s) with an unavoidable drop in context grounding accuracy.

VI. CONCLUSION AND FUTURE WORK

In this work, we explored model parameter size vs. RAG pipeline sophistication. We demonstrated that when utilizing local consumer GPUs, Qwen2.5 0.5B on Modular or Advanced RAG yields the best "sweet spot", achieving highest accuracy(88.02%) with lowest hallucination rate(11.98%) in below 1.9 seconds. Perhaps the most important observation, however, is that post-retrieval reranking is not only an accuracy enhancer, but is a latency accelerator which reduces query size and prevents GPU memory pressure problems. Future research may expand upon this trade-off curve as model quantization is considered (specifically the effect of 4-bit and 8-bit GGUF files).

REFERENCES

- [1] Y. Gao, Y. Xiong, X. Gao, et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2023.
- [2] S. Jeong, J. Baek, S. Cho, et al., "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity," Proceedings of the Association for Computational Linguistics (ACL), 2024.
- [3] Z. Zhang, X. Wang, Y. Jiang, et al., "KBM: Delineating Knowledge Boundary for Adaptive Retrieval in Large Language Models," arXiv preprint arXiv:2402.13253, 2024.
- [4] Z. Yao, W. Qi, L. Pan, et al., "SEAKR: Self-aware Knowledge Retrieval for Adaptive Retrieval Augmented Generation," arXiv preprint arXiv:2403.04562, 2024.
- [5] S. Pandey, "Can Small Language Models Use What They Retrieve? An Empirical Study of Retrieval Utilization Across Model Scale," Empirical Methods in Natural Language Processing (EMNLP), 2026.
- [6] J. Smith, et al., "Optimization of Retrieval-Augmented Generation (RAG) Architectures using Quantized Small Language Models (SLMs): A Performance Analysis," International Journal of Cognitive Computing, 2025.
- [7] T. Chen, et al., "VectorLiteRAG: Latency-Aware and Fine-Grained Resource Partitioning for Efficient RAG," IEEE Transactions on Parallel and Distributed Systems, 2025.